

1 Archive Contents and Version

Mountain Search UAV System | Technical Guide | 28 September 2026
Source revision: UAV-SEARCH-20260928

This archive contains the mobile application, ground station and UAV source code, together with MATLAB simulations of the three search modes, software verification records and outdoor footage. All paths in this guide are relative to the archive root.

File locations

Directory	Contents
code/	Three software components, integration tests and configuration checks
simulation/	MATLAB scripts, elevation data, trajectories, figures and animation
experiments/phone_relay_20260928/	Inputs and results from the local phone-record relay test
experiments/outdoor/	Original outdoor footage, ground-station footage and images
records/	Verification logs, source provenance and file checksums
docs/	This guide in Word, PDF and Markdown formats
LICENSES/	Source licenses and third-party copyright notices

Component directory abbreviations

M, G and U refer to the mobile, ground-station and UAV directories, respectively:

code/mobile_application
code/ground_station
code/search_uav

Current behavior

Mode 1 estimates the trip end time from the planned route and a fixed walking speed. Mode 2 checks whether the latest valid GPS sample exceeds the configured update timeout. Mode 3 places an SOS request in the verification queue. In all three modes, the operator verifies the situation and confirms a search before creating an event, reviewing its route and dispatching the mission.

Application and protocol version numbers are retained. The archive revision is recorded separately in code/source_revision.json. The software retains its existing language options.

2 Local Setup and Verification

Software verification used Python 3.12, Node.js 24 and a C++14 compiler. Simulation recomputation used MATLAB R2025b. Run the following commands from the archive root.

Create the Python environment

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-verification.txt
python scripts/prepare_local.py
```

Install mobile dependencies

```
cd code/mobile_application
npm ci
cd ../../
```

Run the complete verification sequence

```
python scripts/verify.py --output local-results/verification
```

Use a new output directory for each run. The sequence checks lint rules, TypeScript types, version alignment, mobile tests, Python syntax, ground-station and UAV tests, C++ target-state handling and cross-component protocols. It then checks the saved MATLAB trajectories and figure axes. An `all_passed` value of `true` in `summary.json` means that all listed checks passed.

Repeat the local relay test

Install Mosquitto and make the `mosquitto` executable available on the command path. The test starts a temporary broker that listens only on the local loopback address and stops it when the test finishes.

```
python code/integration_tests/run_phone_uav_gs_bench.py \
  --output local-results/phone-relay
```

The test sends a synthetic phone record through a local HTTP interface and uses a running Mosquitto broker for relay and acknowledgement. Its output includes nine key records, the broker log and file checksums.

On Windows, activate Python with `.venv\Scripts\Activate.ps1` in PowerShell. The complete shell-based verification entry point requires Git Bash or a compatible shell. Configure ROS programs on Ubuntu as described in Section 5.

3 Mobile Records and Trip-Time Estimation

Mode 1: trip booking

The user enters a trip name, departure date and departure time, then selects successive locations along the planned route on the map. The interface displays the estimated end date and time automatically. Editing the route, departure date or departure time recalculates the estimate.

The application uses the Haversine formula between consecutive coordinates and sums these distances. It uses a fixed walking speed of 4 km/h and rounds the estimated duration up to a whole minute. The end timestamp is the departure timestamp plus that duration. Dates spanning midnight, a year boundary or a leap day retain the complete end timestamp.

```
duration_minutes = ceil(route_distance_m / 4000 * 60)
end_time = departure_time + duration_minutes * 60 seconds
```

The 4 km/h value follows the general walking-time reference in Ordnance Survey's Map Reading guide. This estimate uses horizontal route distance; selected points should represent the planned route turns. Gradient, ground conditions and stops affect actual duration, so an overdue record still requires operator contact and verification.

Stored field	Meaning
waypoints	Planned route positions in the order selected by the user
expectedEndAtMs	Estimated end timestamp in Unix milliseconds
endDate / endTime	End date and time displayed by the interface
routeDistanceM	Total route distance in metres
estimatedDurationMinutes	Estimated duration, rounded up to whole minutes
walkingSpeedKmh	Fixed value: 4
estimationMethod	route_distance_fixed_speed_v1

Modes 2 and 3

Mode 2 uploads GPS positions and sample timestamps during the hike. The ground station checks the latest valid sample for a timeout and uses the uploaded position history when preparing a flight. Mode 3 stores the location and request information recorded when the user activates SOS, for operator verification and square-spiral route generation.

Implementation: `M/services/eventBookingRecord.ts`. Booking interface: `M/pages/EventBookingPage.tsx`.

Tests: `M/__tests__/eventBookingRecord.test.ts`.

Reference: Ordnance Survey, Map Reading, "Timing", printed page 23. The complete source link is in `docs/Sources.txt`.

4 Ground-Station Verification and Dispatch

Common verification procedure

Overdue bookings, GPS update timeouts and new SOS requests enter the verification queue. The operator reads the user and emergency-contact details, contacts them and records the outcome. A confirmed safe status closes the alert. If safety cannot be confirmed, the operator contacts the available emergency contacts. After completing these records, the operator explicitly confirms a search before the platform creates a search event.

An SOS request no longer creates a dispatchable event directly. Repeated scans or confirmations do not create duplicates. A cancelled SOS cannot be confirmed through a stale alert. An unverified SOS event from the earlier implementation must complete contact verification before route preparation, provided it has not already entered execution.

Select an event, review its route and dispatch

After the operator selects an event, the ground station reads the associated user records and generates a route. Mode 1 uses the planned route; Mode 2 uses GPS history ordered by sample time; Mode 3 generates a square spiral around the SOS location. The operator reviews the route and settings before dispatch. Changes to the relevant records or operation state require the mission to be prepared again.

All three modes have the same initial event priority and the same waiting-time rule. SOS mode alone does not assign the highest priority. The operator selects and dispatches the mission.

Runtime configuration

Environment variable	Purpose
GS_FIREBASE_DATABASE_URL	Database URL shared with the mobile application
GS_FIREBASE_CREDENTIALS	Local service-account file path
GS_T_LOCATION_UPDATE_SECONDS	Positive Mode 2 update timeout, in seconds
GS_T_WAIT_SECONDS	Positive queue waiting-time threshold, in seconds
GS_EVENT_TIMEZONE	Time zone for legacy bookings; template: Asia/Hong_Kong
GS_STATE_DIR	Local directory for dispatch journal files
MQTT_BROKER / MQTT_PORT	Message broker address shared with the UAV

Use `G/runtime.env.example` as the template. Supply and export the actual values, then run `python ground_station.py` from `G`. Both threshold fields are blank in the template. The 1-second GPS threshold in the tests checks time-boundary behavior.

Verification logic: `G/rescue_event_manager.py`. Persistence and mission preparation: `G/rescue_repository.py`, `G/active_event_queue.py` and `G/ground_station.py`.

5 UAV Software and External Flight Stack

Mission reception and execution status

The ground station and UAV exchange missions and status over Wi-Fi using MQTT messages. A mission contains event and execution identifiers, a waypoint sequence, flight altitude, hover duration and return settings. The bridge validates the mission, passes it to the flight program and reports waypoint arrivals and mission stages to the ground station.

The phone-record receiver saves uploads received through its HTTP interface. When the forwarding condition is met, it sends the record to the ground station. The ground station saves the record and returns an acknowledgement matching its identifier and content hash. The UAV then marks that transfer as complete. The recording program saves downward-facing video on the onboard computer.

File within U	Purpose
catkin_ws/src/rescue_bridge/src/mqtt_bridge.py	Mission reception and execution-status exchange
catkin_ws/src/rescue_bridge/src/mission_commander.cpp	Waypoint and return-stage execution
drone_system/receiver/phone_sos_receiver.py	Phone-record reception, storage and forwarding
drone_system/launcher/cam_recorder.py	Onboard video recording
drone_system/launcher/drone_console.py	Onboard program status and settings interface

Pinned external dependency

The flight stack uses Fast-Drone-250 from the original Drone repository at the commit below. Its 979 regular files were compared with the original local copy and matched. The external dependency retains its own licenses and documentation.

b3ac1591b63d15270c9856e7b664da0cf9d3ea48

Download and placement commands are in docs/Flight_Environment_Setup.txt. The original onboard directories are /home/mike/Fast-Drone-250, /home/mike/catkin_ws and /home/mike/drone_system. On a different machine, check directory paths, camera indices, serial ports and sensor calibration.

The ROS environment uses Noetic and catkin. Build the external flight workspace before the rescue_bridge workspace. The default flight altitude in rescue_bridge.launch is the original project setting of 5 m. The 80 m above-ground setting in Section 6 belongs to the separate MATLAB simulation.

Phone-receiver settings are in U/drone_system/config/rescue_receiver.env.example. Set UAV_RESCUE_TOKEN when exposing the interface on the local network. Keep database credentials, runtime tokens and actual user records in their respective deployment environments.

6 Three-Mode MATLAB Terrain Simulation

The simulation uses terrain near Pak Tam Chung, Sai Kung. The three modes share a launch location and a target in the hills, using a planned route, pre-dispatch GPS history and a square spiral around the target, respectively. Each mission includes takeoff, cruise, target-area activity, return and landing.

Setting	Value
Random seed	20260926
Launch location	22.402° N, 114.322° E
Target location	22.40782761° N, 114.35687745° E
Target ground elevation	Approximately 177.44 m
Cruise altitude above ground	80 m, following the terrain
Speed limits	Horizontal: 15 m/s; vertical: 3 m/s
Waypoint hover duration	5 s
Mode 3 spiral	Initial side length: 30 m; increase by 30 m every two segments; stop at the first endpoint at least 200 m from the centre

Run commands

Set the MATLAB current folder to simulation and select the required entry point:

```
run_all('paper')
run_all('simulate')
run_all('video')
```

paper redraws the paper figure from saved trajectories. simulate recomputes all modes and exports figures. video creates the animation and requires ffmpeg for video assembly. The paper figure is written to regenerated/paper; complete trajectories, statistics and animation are stored in output.

Mode	Input waypoints	Target arrival / s	Landing complete / s
Mode 1	3	360.66	715.65
Mode 2	15	409.08	764.07
Mode 3	19	344.99	1008.53

Elevation comes from the Mapzen / Tilezen Skadi N22E114.hgt tile, with a 3601 × 3601 grid at 1 arc-second spacing. Routes and GPS history are synthetic simulation inputs generated with the fixed seed. Times start at simulated takeoff. Parameters, inputs and executed trajectories are in scenario_and_settings.json, the per-mode waypoint CSV files and the per-mode execution_trace.csv files.

The MATLAB run includes 30 assertions. A separate Python verification contains 53 checks covering terrain interpolation, 80 m ground clearance, speed limits, waypoint order, return and landing, and fully boxed axes.

7 Software Verification and Relay Records

Verification ran on 28 September 2026 using macOS arm64, Python 3.12.14 and Node.js 24.19.0. Detailed results are in records/software.log and records/verification_summary.json.

Test group	Tests passed
Mobile application: Jest	229
Ground station	233
UAV phone-record receiver	44
UAV mission bridge	44
Video recorder and recording state	12
Onboard sample data	1
Cross-component protocols and state transitions	61
Total	624

TypeScript type checking, ESLint, Python syntax, dependency-version checks, and compilation and execution of the C++ target-state tests also passed. New cases cover trip-time estimation, midnight rollover, recalculation after route edits, SOS contact steps, cancellation, stale requests, repeated scans and rejection of unverified events at dispatch preparation.

Local phone-record reception and forwarding

A synthetic phone record containing an SOS location and GPS data is sent through local HTTP to the UAV receiver. After the receiver saves it, the test sends a matching mission-stage message to trigger forwarding. The ground station validates and saves the received data, then sends an acknowledgement. The UAV uses that acknowledgement to complete the transfer.

Record numbers	Saved contents
01–03	Phone request, HTTP receipt and record saved by the UAV
04–05	Mission stage triggering forwarding and outgoing MQTT envelope
06–07	Phone record and envelope saved by the ground station
08–09	Ground-station acknowledgement and final UAV delivery state

The request_id, mission_id, record-content hash and envelope hash link these records for comparison. run_summary.json lists the result and test conditions. SHA256SUMS.txt contains the output-file checksums.

These records describe the local software and broker test. Outdoor footage is listed in Section 8. The software tests also cover incorrect or missing acknowledgements, duplicate data and recovery after a restart; the test cases and results are retained in the source files and logs.

8 Outdoor Materials, Sources and Integrity

Outdoor evidence

File in experiments/outdoor/	Contents
outdoor_flight.mp4	Original complete outdoor-test video
ground_station.mp4	Original ground-station operation video
airframe.jpg	UAV hardware photograph
launch_area.png	Original frame showing the launch area
search_area.png	Original frame showing the search area

The original video files are preserved byte for byte. Some demonstration footage has been sped up; time labels in those frames are not measurements of actual mission duration. The images show the hardware, site and search views. Numerical simulation trajectories and timestamps are stored in simulation/output.

Source provenance

The mobile application, ground station and UAV software originate from the FYP projects and subsequent local revisions. Upstream repository links, remote commit identifiers, local input files and checksums are recorded in records/source_provenance.json. The current implementation is recorded in code/source_revision.json. Remote commits and local revisions are identified separately.

The local input revision is WCNC-GPS-FRESHNESS-20260916. The current revision adds route-based trip-time estimation, operator verification before SOS event creation, and equal initial event priority across all three modes. Simulation source and data checksums are also recorded in simulation/verification/source_manifest.json.

Check archive integrity

```
python scripts/check_archive.py
```

This command reads records/file_manifest.json and checks each listed file's SHA-256 checksum and size. If source code, parameters or results are changed, save a new verification record. Preserve the original timestamps and identifiers of existing test outputs.

References

Ordnance Survey's Map Reading guide provides the walking-time reference. Mapzen / Tilezen Skadi provides the elevation tile. The three original FYP GitHub repositories identify the upstream projects. Complete links are in docs/Sources.txt.